

DEVELOPMENT OF HIGH-ACCURACY ROOT PHENOTYPING SOFTWARE

A Project

Presented to the

Faculty of

California State Polytechnic University, Pomona

In Partial Fulfillment

Of the Requirements for the Degree

Master of Science

In

Computer Science

By

John Otters

2024

COMMITTEE MEMBERSHIP

PROJECT: DEVELOPMENT OF HIGH ACCURACY ROOT
PHENOTYPING SOFTWARE

AUTHOR: John Otters

DATE SUBMITTED: Spring 2024

Department of Computer Science

Dr. Amar Raheja
Project Committee Chair
Professor of Computer Science

Dr. Abdelfattah Amamra
Professor of Computer Science

ABSTRACT

The development of agriculture has been instrumental in the success of our species. Progress was initially made slowly, over the course of multiple generations. This was due to the random and uncontrolled nature of early humanity's experimentation. With the advent and adoption of science, advancements have been made at an exponential pace. However, in order to properly conduct a scientific experiment, it is necessary to accurately quantify its results. This is an expensive, time-consuming, and tedious undertaking. This paper details several existing methods of root phenotyping, which is the measurement of a root's physical characteristics, and compares them to a newly developed program.

The program is designed to work with large amounts of human input. It facilitates this by working in multiple stages, each with specific roles and several settings for the user to adjust. It works quickly despite the number of settings, since it saves and loads them for each dataset. It also incorporates a neural network that can be progressively trained and whose predictions can be manually corrected. This lowers the accuracy barrier for the neural network to be useful, and thus requires far less training data. Since the purpose of root phenotyping is scientific experimentation, the program is designed to prioritize accuracy over all else.

TABLE OF CONTENTS

COMMITTEE MEMBERSHIP.....	ii
ABSTRACT.....	iii
LIST OF TABLES.....	vii
LIST OF FIGURES.....	viii
CHAPTER 1: INTRODUCTION.....	1
CHAPTER 2: LITERATURE SURVEY.....	2
2.1 SmartRoot.....	2
2.2 RootNav 1.....	5
2.3 RootNav 2.....	7
2.3.1 Neural Network Definition.....	7
2.3.2 Convolutional Neural Network Definition.....	8
2.3.3 Encoder-Decoder Definition.....	9
2.3.4 RootNav 2 Design.....	10
2.3.5 RootNav 2 Results.....	11
2.4 RootNet.....	11
2.4.1 RootNet First Channel.....	12
2.4.2 RootNet Second Channel.....	13
2.4.3 RootNet Results.....	14
CHAPTER 3: PROJECT GOAL.....	15
3.1 Accuracy.....	15
3.2 User Time.....	15
3.3 Adaptability.....	16

3.4 Fully- or Semi-Automated Design.....	16
3.4.1 Accuracy.....	16
3.4.2 User Time.....	17
3.4.3 Adaptability.....	17
3.4.4 Decision.....	18
3.5 Output Format.....	18
CHAPTER 4: METHODOLOGY.....	18
4.1 Program Stages.....	20
4.1.1 Gaussian Filter.....	20
4.1.2 Label Cropping.....	21
4.1.3 Edge Finder.....	22
4.1.4 Texture Matcher.....	24
4.1.5 Fill Small Holes.....	26
4.1.6 Connect Separated Roots.....	27
4.1.7 Remove Small Objects.....	30
4.1.8 Manual Correction.....	31
4.1.8.1 Delete in Radius.....	31
4.1.8.2 Delete Segment.....	32
4.1.8.3 Add Segment.....	33
4.1.9 Neural Network.....	36
4.1.9.1 Network Structure.....	36
4.1.9.2 Network Integration.....	39
CHAPTER 5: EVALUATION.....	39

5.1 Datasets.....	40
5.2 Metrics.....	43
5.3 Results.....	44
5.4 Potential Issues.....	45
5.5 Program Strengths.....	46
5.6 Network Training.....	47
CHAPTER 6: FUTURE WORK.....	48
CHAPTER 7: CONCLUSION.....	48
REFERENCES.....	50

LIST OF TABLES

Table 1: Comparison of software accuracy.....	44
Table 2: Comparison of software speed.....	44

LIST OF FIGURES

Figure 1: SmartRoot scanning process.....	3
Figure 2: SmartRoot next node selection process.....	4
Figure 3: Fully connected neural network.....	8
Figure 4: Convolutional layer.....	9
Figure 5: RootNav 2 network structure.....	11
Figure 6: U-Net architecture.....	13
Figure 7: RootNet network structure.....	14
Figure 8: Gaussian filter stage.....	21
Figure 9: Label cropping stage.....	22
Figure 10: Probe movement options.....	23
Figure 11: Edge finder stage results.....	24
Figure 12: Texture matching stage results.....	26
Figure 13: Effects of the fill small holes stage.....	27
Figure 14: Connect separated roots stage results.....	30
Figure 15: Effects of the small objects stage.....	31
Figure 16: Effects of the remove segment algorithm.....	33
Figure 17: Example of the add segment algorithm.....	35
Figure 18: Dense block used in the neural network.....	37
Figure 19: Sample image from main dataset.....	40
Figure 20: Sample image from the rapeseed dataset.....	41
Figure 21: Sample image from the Arabidopsis dataset.....	42
Figure 22: Example of errors in preexisting rapeseed labels.....	42

Figure 23: Example of errors in preexisting Arabidopsis labels.....	43
Figure 24: Example types of errors made by RootNav.....	46

CHAPTER 1: INTRODUCTION

The discovery and adoption of agriculture revolutionized the lives of early mankind, providing a consistent and plentiful food source. The need to tend crops forced humans to stay in the same location for long periods of time, which caused the first villages to form. With the increased food supply, humans were able to spend less time looking for food and more time on other pursuits [1]. This was the driving factor in the progress of civilization.

In the modern day, agriculture still plays a massive role in our lives. For example, almost everything we eat comes from plants, either directly or as food for livestock. Most fabrics, such as wool or cotton, also come from agriculture. Due to the powerful benefits it provides, there is a strong incentive to improve the output, efficiency, and consistency of agriculture.

Early farmers attempted to improve yields by only breeding the largest animals or by planting their best performing crops. This technique worked, but it was slow since farmers had to wait an entire year to determine which crops performed best. The approach was also unscientific, due to not taking weather, soil conditions, or other external factors into account. To effectively test plant phenotypes, these and all other variables must be controlled across tests.

One effective modern method for controlling variables is plant tissue cultivation. This method involves the growth of plant cells in an artificial culture medium under highly controlled and sterilized conditions [2]. It involves removing a portion of a grown plant, sterilizing it, and letting it regrow in laboratory conditions. The resulting plant will be a genetic match of the source plant. This process is significantly faster than growing a new plant from a seed. It can also be done using a clear liquid as a medium instead of soil, making it easy to view the plant's growth. This is especially important, since the sample's progress must be regularly measured for the experiment to produce any useful results.

One common metric of growth is the size of the sample. There are several methods to measure a sample's size, but they all have drawbacks. Measuring the sample's length is difficult unless the sample is perfectly straight, and physically straightening it would interfere with the plant's natural growth. Weighing the sample is another option, but it is also difficult to do without disturbing the plant's growth. Both methods are also very time consuming to do manually, which is the most limiting factor to performing tests at scale.

As high-quality cameras and powerful computers have continued to decrease in price, more researchers are turning to software to automate this process. A root phenotyping software is broadly categorized as either semi-automated or fully-automated. Semi-automated software requires some human input, while fully-automated software is able to label a sample's phenotypes without any human involvement. Semi-automated software is slower than fully-automated, and requires a skilled human labeler, but generally produces significantly more accurate results. Additionally, since they are controlled by a human, they can produce good results on a wide variety of samples they have not been trained on or designed for.

CHAPTER 2: LITERATURE SURVEY

Due to the importance of the field, there have been many different attempts to automate root phenotyping. The following are a representation of the progress made over time and the different techniques employed.

2.1 SmartRoot

SmartRoot is a semi-automated tool that can label plant roots [3]. It requires a starting position on the root from a human user, and then attempts to trace up and down the root. It uses

traditional methods to represent the root as a series of connected nodes. The nodes have different sizes depending on the thickness of the root at that position. SmartRoot can automatically determine the color of the root and the background from the initial position. It can also deal with variable lighting across the image because it adjusts itself after each node.

SmartRoot records the pixel color at the initial position and then scans in multiple directions. The scanning process continues until it finds a pixel that is sufficiently different from the starting pixel. These scan lines are treated as candidates for the width of the root at that location. After performing the scans, the shortest line is considered to be the width of the root, and it assumes the other lines traveled down the length of the root. It uses the center of the width line as the center of the first node, and the diameter of this node is the length of the line. SmartRoot also up-samples the image using interpolation to make this operation more precise.

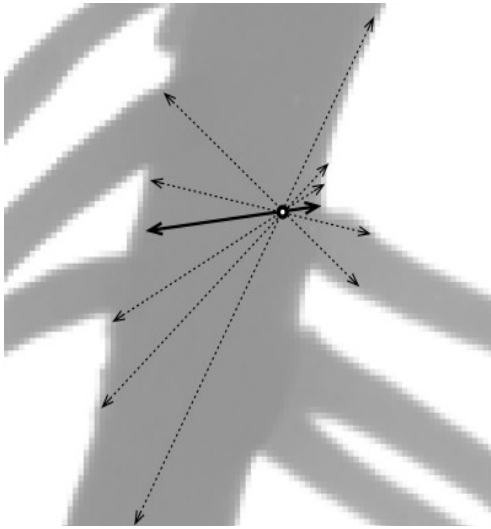


Figure 1: Illustrates the scanning process that estimates the local root width. The lines represent the different scan trajectories centered on the user's input. The solid line is the shortest found and assumed to be the root's width. Source: Adapted from [3]

After finding the first node, SmartRoot searches up and down the root, where up and down is perpendicular to the width line. To find the next node, SmartRoot searches along a 90°

arc with a radius double the diameter of the current node. It searches for a circle of pixels with a color and diameter similar to the current node. If they are a significantly different color, it assumes it is not a root. If the circle is significantly bigger or smaller, it assumes they belong to a different root, since roots do not tend to quickly change size. If it finds a match, it marks the new node and repeats the search. If it fails, it will search again with a 120° arc with half the radius as before. This is to catch the one exception to roots quickly changing size, which is a thick root with a conical tip. If this check fails, SmartRoot marks the current node as the end of the root and stops its search.

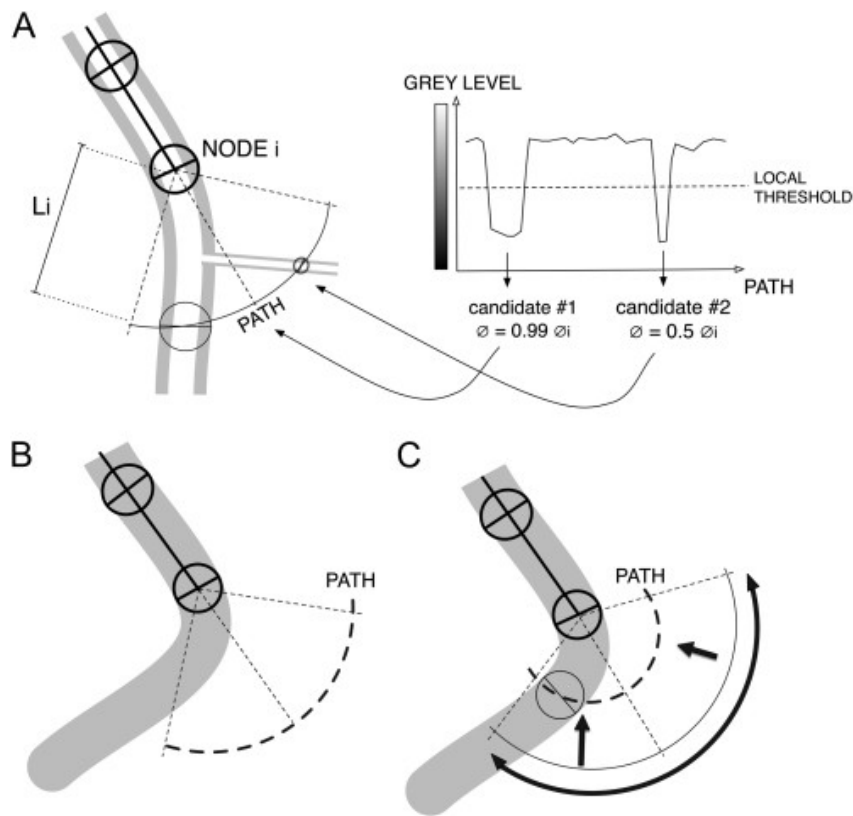


Figure 2: Illustrates how new nodes are found along the root. A, Pixel values are evaluated along a 90° arc from the previous node. B, Demonstrates why the algorithm fails when the root curvature is too strong. C, The algorithm performs a second sweep across an arc with a smaller radius and larger angle.
Source: Adapted from [3]

SmartRoot is much more effective than tracing by hand, but it suffers from accuracy issues. These inaccuracies can be corrected by a human, but this is a very slow process, and the human may miss some errors. SmartRoot is inherently limited since it only takes local information into account. Even humans can find it difficult to identify roots without looking at a larger portion of the image, so tools that use global information can achieve better results.

2.2 RootNav 1

Like SmartRoot, RootNav 1.0 is a semi-automated phenotyping tool that labels plant roots [4]. Unlike SmartRoot, RootNav uses global information to make its classifications. SmartRoot classifies pixels based on binary thresholds, but RootNav uses Expectation Maximization (EM) clustering to assign pixels a probability. It then uses A* to find the best path from root tips to the seed, using the probabilities as the costs for each edge. This allows the tool to take more of the image into account before making its final judgment. The seed and root tip locations are given by the user. The user can also enter any number of “waypoints” that the path must travel through as a form of error correction. RootNav’s name comes from this similarity to GPS pathfinding algorithms.

Most phenotyping tools use binary thresholding, where a pixel’s color is used to make an immediate, binary classification on whether it is a root. This approach suffers when the image has variable lighting, and even adaptive thresholding still ignores the structure of the root. RootNav 1.0 uses local EM clustering across the image to find probable root locations. Local EM clustering is an iterative method that attempts to create an optimal probability model given a set of input data. The method also assumes that there is hidden information that is necessary to find the perfect solution. It begins by creating a set of variables that represent the hidden

information and randomizing the parameters of the probability function. At each iteration, it uses the probability function to obtain the probability of the known inputs and the assumed values of the hidden variables. It then replaces the hidden variable values and uses them to update the probability function parameters. It repeats these steps until the variables converge. The advantage of this method is it only assigns the pixels a probability of belonging to a root, allowing future processes to consider the root's structural information. Since RootNav uses local EM clustering, the algorithm is run multiple times across sections of the image. This acts like adaptive thresholding and gives some resistance to dynamic lighting.

After obtaining root probabilities, RootNav transforms the image into a graph, where all the pixels are connected to their 8-neighbors. The cost of each edge is the probability that a pixel is not a root, multiplied by $\sqrt{2}$ for diagonal neighbors, due to the larger distance. RootNav will then find the best paths between each seed and all their root tips using A*. The A* algorithm uses a distance heuristic that assigns an extra cost the farther a pixel is from the destination. This heuristic does not help much if there is a clear path from the tip to the seed, but if there are breaks due to obstructions or poor image quality, it greatly helps keep the algorithm from wandering the image. Another heuristic is used to combat issues faced when tracing curved roots. The algorithm tends to find the shortest path, so a sufficiently thick curved root will tend to be traced along the inner curve, rather than the center. The heuristic penalizes nodes based on their distance to the root's center of mass, encouraging paths along the middle of the root. After the algorithm runs, the user can place "waypoints" that act as intermediate stops. The algorithm will start at a root tip, find all the waypoints, and then find the seed. A* allows RootNav to use the structure of a root, as well as local pixel colorings. It also eliminates false positives that are far away from the root since the pathfinding algorithm will never reach those positions.

RootNav 1 is significantly better than SmartRoot in every way. However, it still suffers from accuracy issues, which also require human intervention to fix. It is still useful in producing high accuracy data to train neural networks, or small-scale analysis, but it is not fast enough in practice for large-scale testing.

2.3 RootNav 2

RootNav 2 builds off RootNav 1 by using neural networks to obtain information that used to be given by a human [5]. The network produces a segmentation of the background, first-order, and second-order roots. The segmentation is used as a probability map of root locations for the A* algorithm, replacing the expectation maximization algorithm from RootNav 1. The network also produces a heatmap regression of likely root tip locations. The heatmaps are post processed using non-maximal suppression, and the final locations are used as the starting points for the A* algorithm. These points were obtained from a human user in RootNav 1. From there, the tool is the same as RootNav 1.

2.3.1 Neural Network Definition

A standard neural network performs transformations on input data using a series of layers. Each layer uses a set of “neurons” that are conceptually based on biological neurons found in animal brains. These neurons are “connected” to each piece of the input data, where each piece is a real number. For example, color images are represented by a 3D array, where the first two dimensions are the spatial coordinates of each pixel, and the third dimension is the RGB values. Each neuron has a different coefficient, or weight, associated for each piece of the input. Each input-weight pair is multiplied together, and the results are summed. The sum is then run

through a non-linear “activation function”, which produces the neuron’s final “activation”. Since each neuron has their own weight, they will each produce different results depending on if the input data contains specific features that they value more heavily. The outputs of each neuron are then treated as the inputs for the next layer of neurons, and this repeats until the final, or output, layer. The neurons in this layer typically have a sigmoid activation function, which constrains the neuron’s output to be between 0 and 1. Activations greater or equal to 0.5 are considered a positive classification, while anything less is considered a negative. For image segmentation, the output layer consists of a 2D array of neurons, each representing an individual pixel from the image. A positive classification means the network has detected a root on that pixel, otherwise it is considered to be part of the background.

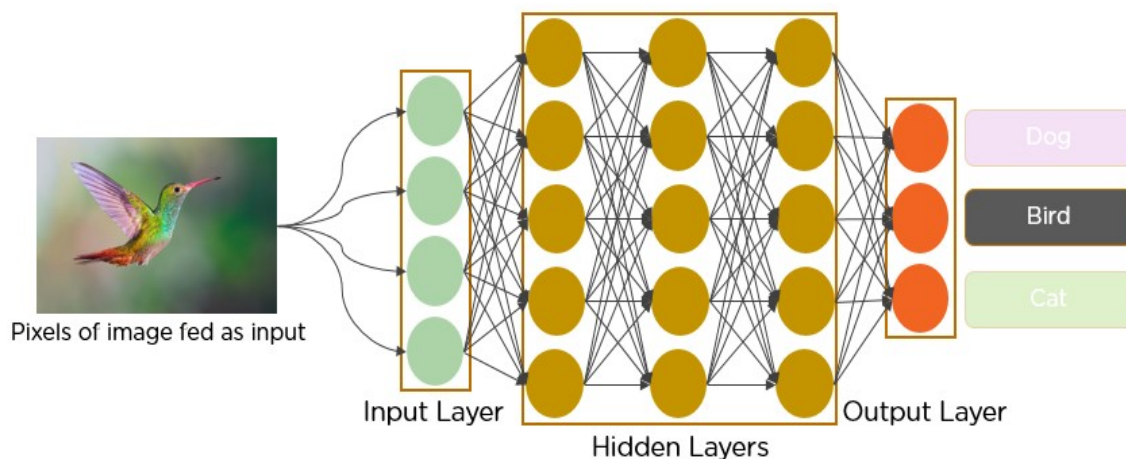


Figure 3: Illustrates fully connected network structure. Input is transformed by each layer until produced by output. Each layer uses the output of the previous layer as input. Each neuron is connected to each input of the previous layer. Each neuron of the output represents a possible classification. The output neuron with the highest activation is the network’s prediction.

Source: Adapted from [6]

2.3.2 Convolutional Neural Network Definition

Standard neural networks are able to perform image segmentation, but they are not well suited for it. These networks are “fully connected”, meaning each neuron is connected to each input. For images that contain millions of pixels, this quickly becomes computationally

intractable. In addition, the network's structure does not lend well to image processing. Each neuron has specific weights for each input, meaning that they learn features by position. If any feature in an image was shifted by a single pixel, it would become completely unrecognizable. Convolutional neural networks were developed to combat these issues. They consist of a small set of neurons, called the kernel, which is repeatedly used on different local sections of the image. Fully connected networks look at all the data at once and try to find global features, while convolutional networks look at small sections of the data at once and try to find local features. Since images tend to mostly contain localized features, convolutional networks can trim a significant portion of the search space with little impact on accuracy. This allows the network to learn more effectively, as well as considerably reducing the computational requirements.

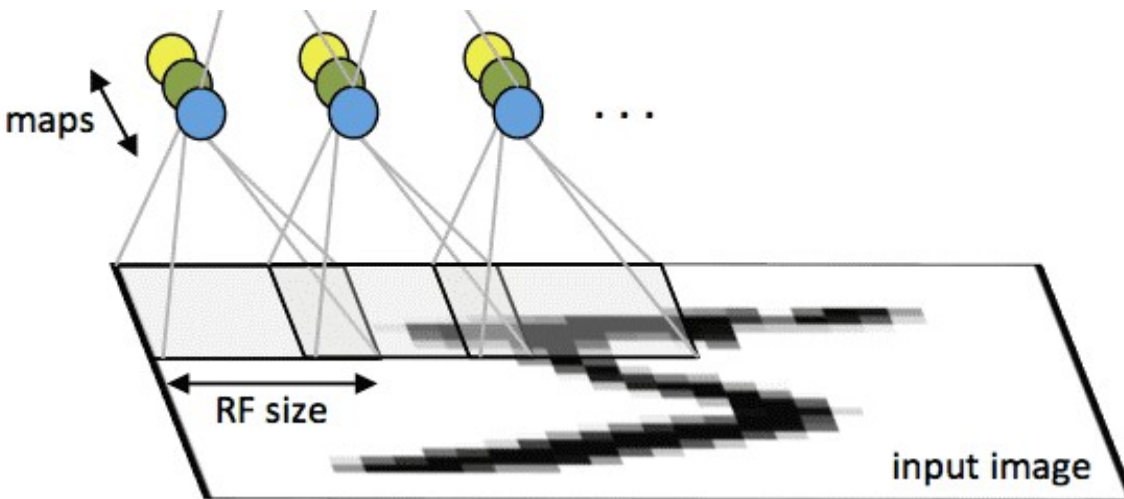


Figure 4: Illustrates how the kernel checks each section of the image for specific features. The kernel keeps the same weights for every section it checks.

Source: Adapted from [7]

2.3.3 Encoder-Decoder Definition

The network used in RootNav 2 is a convolutional encoder-decoder. The main weakness of convolutional layers is that they can only take local information into account. An encoder-

decoder model is a two-stage design that compresses and then processes information. The encoder portion gradually shrinks the size of the input using a sequence of blocks comprised of convolutional layers followed by pooling layers. A pooling layer takes groups of neighboring pixels, usually 2x2, and produces a single average or maximum. The result is an image with a quarter of the original pixels, which greatly compresses the data while minimizing information loss. This allows subsequent convolutional layers to utilize information from a larger effective area. After several encoding blocks, the encoder gives the compressed information to the decoder.

The decoder also consists of several blocks, each corresponding to an encoder block. The decoder blocks are comprised of convolutional layers followed by a deconvolution layer. This layer uses learned weights to upsample the data. Once the data is the desired size, a final layer produces a segmentation. Where the encoder's job is to identify the most important information and discard everything else, the decoder's job is to process that information to produce the desired result.

2.3.4 RootNav 2 Design

The encoder-decoder model in RootNav 2 takes images of 1024x1024 pixels and produces segmentations and regression maps of 512x512 pixels. The encoder reduces the data to a 128x128 pixel feature map. The decoder takes this as input and produces a 512x512 feature map. This map is then processed using two separate branches: one for the segmentations and one for the heat maps. Each branch uses its own series of 1x1 convolutional layers, with the segmentation branch finishing with a sigmoid layer. These outputs are used to supply the

information that would have been given by the user in RootNav 1. This allows the software to become fully-automated.

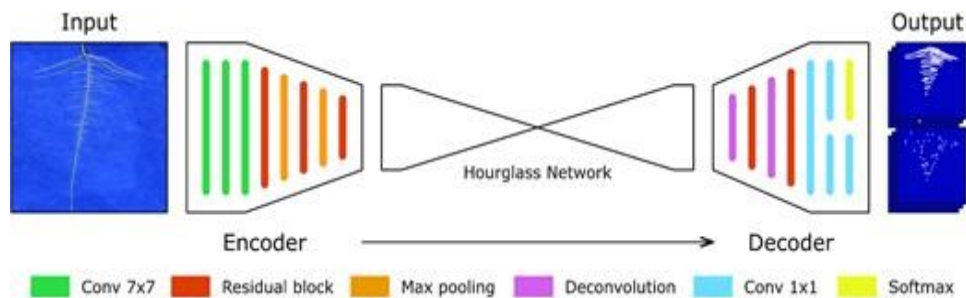


Figure 5: Illustrates the structure of the network. The output has two branches: one for segmentation, one for feature heatmaps.
Source: Adapted from [5]

2.3.5 RootNav 2 Results

RootNav 2 is less accurate overall than the first version due to not utilizing human input, but it is immensely faster. Its main accuracy weakness comes from memory limitations, which limit the size and complexity of the network. Image segmentation requires a large amount of GPU memory because each pixel component is an input variable. This is especially pertinent for root phenotyping because of the high resolutions required to capture a root's fine details. One method of reducing memory consumption is splitting an image into separate tiles and processing them individually. However, this approach loses global information, which is vital for root phenotyping. The creators decided to cut down the network's trainable parameters and depth to allow for 1024x1024 color images. Even with this sacrifice, they were still required to down-sample many of their images.

2.4 RootNet

The current state of the art is a tool called RootNet [8]. Unlike the previous tools, RootNet is completely neural network based. RootNet is designed to solve the memory constraint issues that plagued RootNav 2 and analyze root images at high resolutions without losing global information. To identify fine details, the image must be analyzed at the highest resolution possible. RootNav 2 compensated for this by reducing the size of the network, but this hurts the network's problem-solving ability. RootNet solves this issue by using a dual channel approach. The first channel obtains fine details and the second obtains global information. The results are then combined in the feature fusion module, which is another neural network that produces the final segmentation.

2.4.1 RootNet First Channel

The first channel is a convolutional U-Net. A U-Net is a type of encoder-decoder model that is designed to minimize the information loss caused by the encoder. An encoder's purpose is to reduce the size of the data while retaining as much useful information as possible, which makes it easier for the decoder to process. However, the encoder will always discard some useful information. A U-Net fixes this by propagating the output of each encoder block to the corresponding decoder block. This provides the decoder with important context that allows it to make more informed decisions. For technical reasons it also makes the network more resilient and easier to train.

This channel is run on patches of the full resolution image. The results of all the runs are combined into a single output with the same size as the original image. This output is then combined with the output of the second channel in the feature fusion module to produce a final

result. This approach is necessary since U-Nets require much more memory to run since the decoder propagates its intermediate outputs. By splitting the image into patches, the network can identify fine details without compromising on its size.

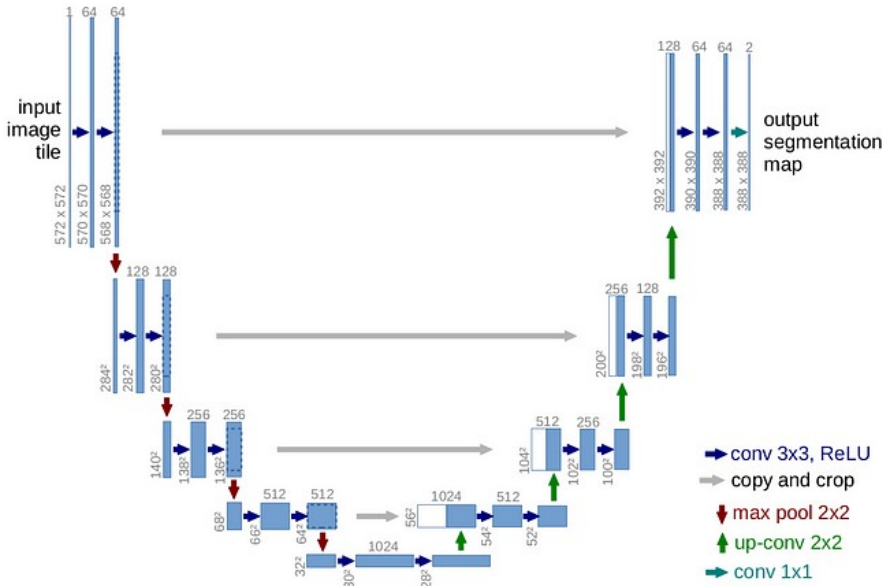


Figure 6: Illustrates the U-Net architecture. Each encoder block contains two convolutional layers followed by a maximum pooling layer. Each decoder block contains two convolutional layers followed by an upsampling convolutional layer. The results of each encoder block are passed to the corresponding decoder block before they are downsampled.

Source: Adapted from [9]

2.4.2 RootNet Second Channel

The second channel is a basic convolutional network that consists of a series of convolutional blocks. It is run on a downsampled version of the image and produces information that is fed to the feature fusion module. The focus of this channel is to capture global information at the expense of fine details. This channel is significantly smaller and less complex than the first channel, as its job is simply to obtain global structural information.

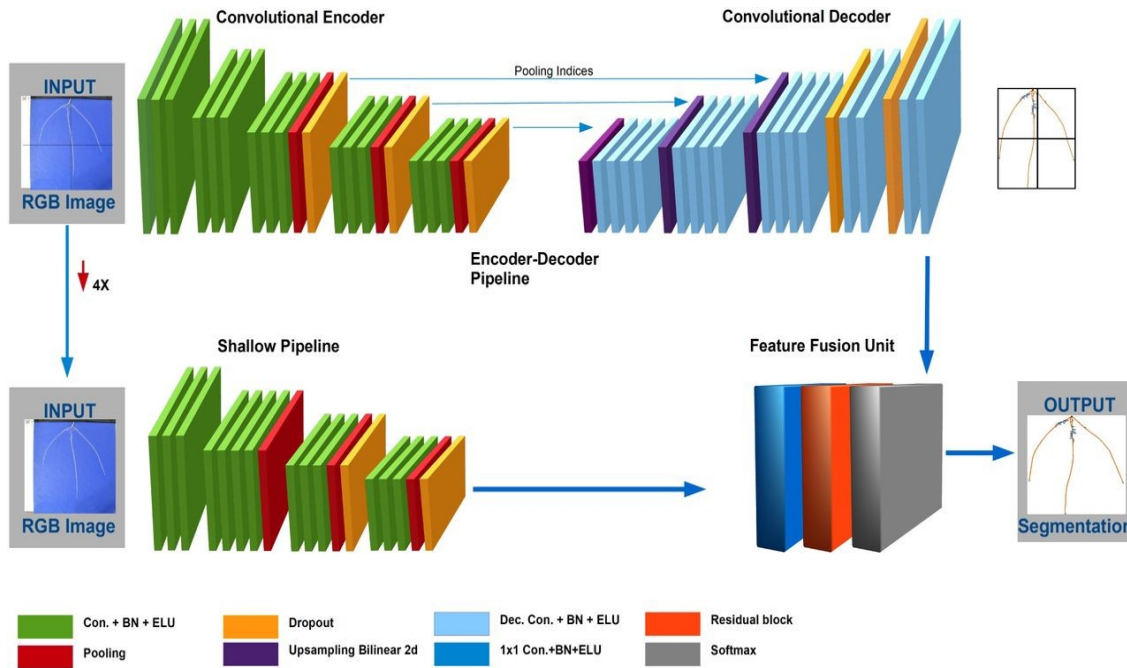


Figure 7: RootNet’s architecture. The top network is the first channel. Note the “U” shape of the U-Net. It is run multiple times; one for each division of the full resolution image. The bottom network is the second channel. It is run once on a downsampled version of the entire image. The results are combined in the feature fusion unit, which produces the final segmentation.

Source: Adapted from [8]

2.4.3 RootNet Results

Despite being the most accurate phenotyping tool to date, RootNet still has some significant flaws that are typical of neural networks and fully-automated systems. Firstly, RootNet is only able to run on a small number of plant species, due to the limited number of open datasets available to train it. Secondly, despite being fully-automated in principal, some form of human oversight is necessary to confirm the results are reasonably accurate. Finally, since RootNet is fully-automated, it is impossible to easily correct mistakes. Results with large mistakes must be thrown away and relabeled with some other method, even if the rest of the labels are correct.

CHAPTER 3: PROJECT GOAL

3.1 Accuracy

The main goal of this project is to create root labeling software that produces very high accuracy labels. The software will produce these labels from an image of one or more plants, with the expectation that the image is of at least decent quality. The project's main goal is accuracy because the purpose of root phenotyping is to test the results of scientific experiments. If the labels are inaccurate, the results of the experiment will also be inaccurate. Therefore, the only useful phenotyping software is one that does not make significant mistakes.

3.2 User Time

The secondary project goal is to minimize how many man-hours it takes the software to produce results. Evaluating the results of root phenotyping experiments requires a large amount of skilled human time, and therefore money. This expense can represent a significant portion of the total cost of the experiment. By reducing how long it takes to label each sample, experiments can be performed more cheaply, or with more samples or better equipment. User time is anything in the labeling process that requires human attention. This includes the time it takes to produce labels with semi-automated software, the time it takes to verify the results of fully-automated software, and the time it takes to test and optimize the software on a particular dataset. The amount of computer time is not significant, because computers are significantly faster, cheaper, more scalable, and can work around the clock.

3.3 Adaptability

The final goal is to make the software as adaptable as possible across all possible datasets. A software is adaptable if it can consistently perform well on datasets it was not trained or optimized on. Each dataset contains many different variables, from the species of plant, the image resolution, lighting, background color, and more. Every software will inevitably be better or worse suited to specific scenarios, but the goal is to minimize these differences. Consistency makes the software useful in more situations and reduces the risk that it will perform poorly on a specific dataset. This makes it safer to commit the time to pick up and learn.

3.4 Fully- or Semi-Automated Design

The most fundamental aspect of designing root phenotyping software is determining how much human interaction it should have. The pros and cons of each approach with respect to the project goals are analyzed below.

3.4.1 Accuracy

Currently, fully-automated software does not have human-level accuracy. It is probable that they will reach and exceed humans eventually, but this likely will not happen in the near future. In addition to having lower accuracy, fully-automated software typically do not have ways to manually correct mistakes, so humans cannot improve the results after the fact.

Semi-automated software is designed from the start to incorporate human input. Humans are very skilled at identifying root structures but are unable to quickly transcribe this information. Semi-automated software receives partial structural information and extrapolates from there. The software may even have ways for users to correct any mistakes it makes,

essentially giving it near human-level accuracy. Because of this, semi-automated software is significantly more accurate than fully-automated software.

3.4.2 User Time

The main advantage of fully-automated software is its ability to produce results without human involvement. This allows them to label an effectively infinite number of samples with no time cost. However, unless the use-case does not have accuracy requirements, humans are still required to verify at least some of the results. Additionally, if significant mistakes are found, they cannot be easily salvaged, meaning the entire label must be thrown out and redone with another method. These issues cut into the potential time fully-automated software would save.

Fully-automated software typically require more time to adapt to a new dataset. Because they operate without human intervention, they must be perfect from start to finish. Meanwhile, semi-automated software can make mistakes with the expectation that the user will correct them. In fact, it is usually not worth optimizing them to the same degree due to diminishing returns. These limitations reduce fully-automated software's theoretical advantages, but they are still significantly faster than semi-automated software.

3.4.3 Adaptability

Currently, humans can generalize significantly better than computers. This means software has significantly lower performance on datasets it was not explicitly trained on. Optimizing or training fully-automated software requires a human labeled dataset regardless. The more control the user has, the less work needs to be done to optimize the software across datasets.

3.4.4 Decision

A semi-automated approach is better for this project's goals as it is more accurate and adaptable. The main drawback, the larger human time cost, is offset by the fact the results do not need to be verified after labeling. Even fully-automated tools still required human labels to be trained and verified. By purposely incorporating a large amount of user input, this project will sidestep these issues and be able to produce those high quality labels for scientific evaluation and training other software.

3.5 Output Format

Many semi-automated software records detailed information about the root, such as 3D information and distinguishing between the main root and offshoots. This approach can be effective since the user can provide so much information. However, this hurts the program's overall accuracy, requires more user time, and is overkill for many experimental evaluations. Therefore, this project will focus on producing a binary segmentation mask. For each pixel, the mask will be True if it represents a root, and False otherwise. This approach is simple, still provides a large amount of detail, and is compatible with many image processing algorithms. In the future, this or other software could use the segmentation as input to create more detailed results, similar to RootNav 2's approach.

CHAPTER 4: METHODOLOGY

The program was designed to work in several stages. Each stage is meant to function independently and can be turned on or off as needed. Each stage also allows the user to customize settings or otherwise provide input. This approach maximizes flexibility and user

control. All settings should be experimented with whenever starting a new dataset for optimal results.

Upon starting the program, it looks inside image and label directories provided by the user. It then sorts the list of image names and loads the first one that does not have a corresponding label. The program generates a window, with the image and controls displayed. The image can be zoomed in and out with the mouse wheel and panned by clicking and dragging. Labels will be shown in different colors depending on the current stage. The program has full undo and redo functionality, with Z and Shift Z respectively, which applies to every state change. There is also a control panel on the right, which provides information on the current stage, settings the user can modify, an option to invert the image's brightness, and information about the pixel under the mouse cursor. After changing a setting, the user can update the labels by pressing Enter. The labels' visibility can be toggled by pressing T. The user can advance to the next stage by pressing N and can return to a previous stage using undo. Lastly, the user can save their results by pressing N on the final stage.

The program is mainly written in Python, with several functions written in C++ for speed. Several stages also take advantage of parallelization, so a CPU with more cores will perform better. A top tier CPU is not required, however. The program was tested and is reasonably responsive on a mid-range CPU. The program is compiled using PyInstaller to function as a more portable Windows executable.

4.1 Program Stages

4.1.1 Gaussian Filter

The first stage is intended to produce an initial set of root candidate pixels. It is supposed to quickly select all or nearly all legitimate roots, while still eliminating a large number of background pixels. This is an excellent starting point and reduces the computational requirements of some future stages. The stage accomplishes this with a gaussian blur followed by a threshold filter. A gaussian blur is a weighted average of the pixel brightnesses surrounding a central pixel. The closer to the center a pixel is, the higher its weight. The blur is applied to each pixel of the image and is then compared to the original image. If the original pixel is brighter than a multiple of its corresponding blurred pixel, it is considered to be a root candidate.

This approach works by assuming that roots are brighter than their neighboring background. Since a blurred pixel is the average of the surrounding pixels, it can be considered a representation of the neighborhood brightness. If the original pixel is brighter than this representation, it is also likely brighter than its background, and thus has a significant chance of being a root. This method selects almost all roots, but also a large amount of background pixels. Importantly, background pixels adjacent to roots are not selected, because they are overpowered by the root's brightness.

The user can adjust the gaussian's sigma and the threshold multiplier. The sigma is effectively the gaussian's radius, and the threshold multiplier represents how much brighter the original pixel needs to be. The sigma should be large enough to allow pixels in the middle of a root to "see" enough of the background, or else the labels will be spotty. It should also be small enough to allow roots in dark regions to not be overpowered by nearby bright regions, and to

prevent bright background next to roots from being selected. The threshold multiplier should be as high as possible while still selecting every root, as this minimizes false candidates.

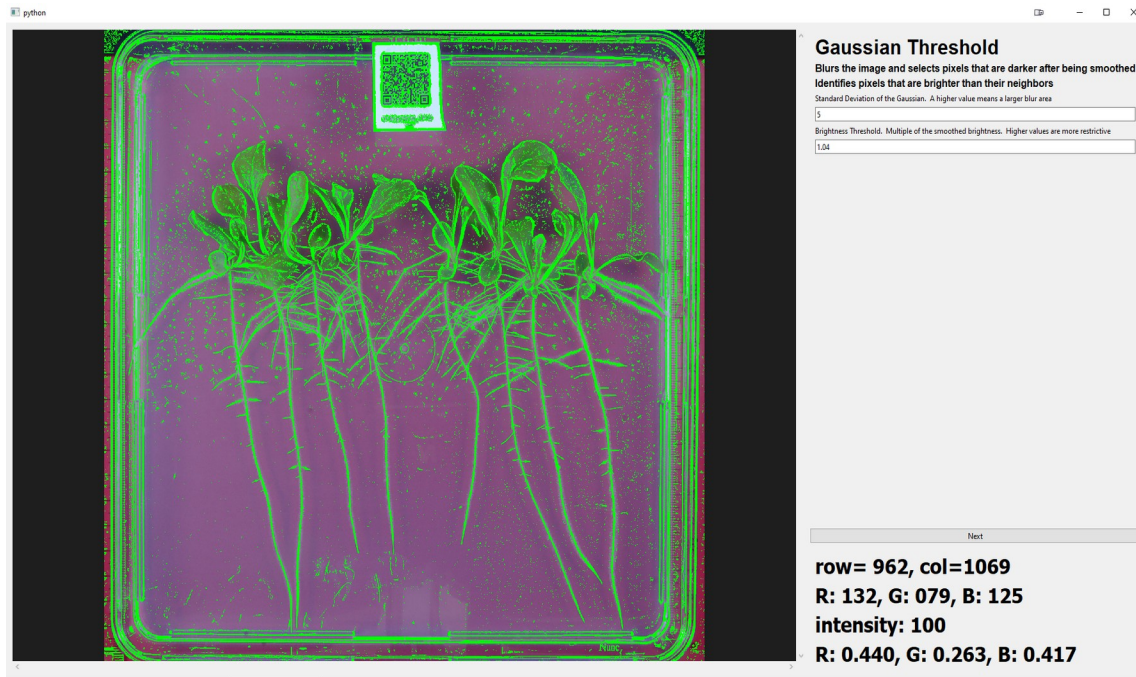


Figure 8: The gaussian filter stage. Root candidates are labeled green.

4.1.2 Label Cropping

The second stage is meant to quickly crop out areas that do not contain any roots. The user clicks on the image to produce a series of points. The points form the outline of a polygon, which is shown in red. When the user presses Enter, all candidates outside the polygon are removed. The user can also hold Shift when pressing Enter, which will instead remove all candidates inside the polygon. This stage is particularly effective when all of the roots are contained in a subset of the image.

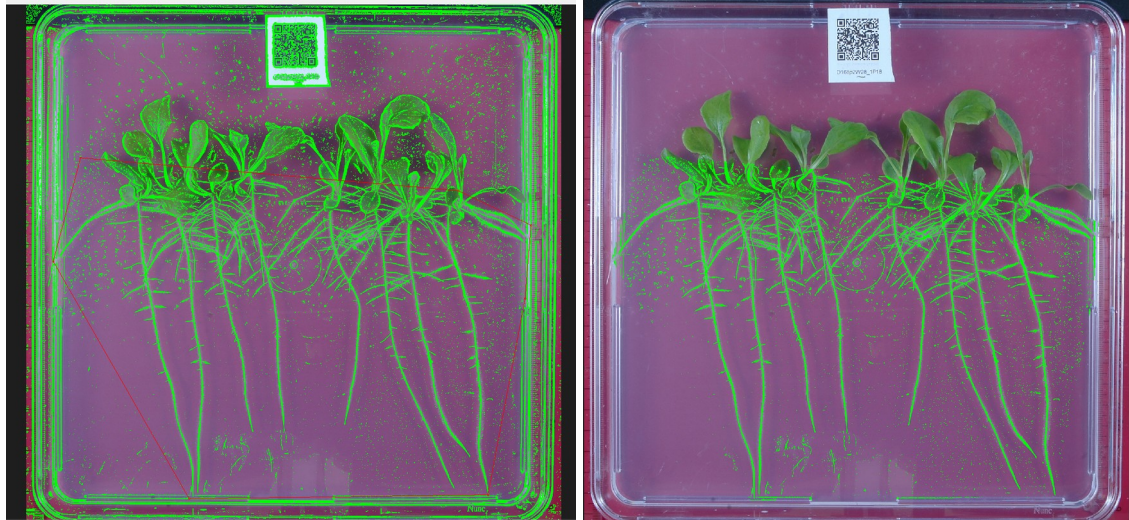


Figure 9: On the left, the polygon drawn around the root boundaries. On the right, the cropped labels.

4.1.3 Edge Finder

The third stage is meant to remove root “fuzz”. Roots often have small offshoots called hairs that can spread to be far larger than the actual root. Root hairs are brighter than the normal background, so the gaussian filter stage tends to select these along with the root’s real body, which negatively affects the overall results. The goal of this algorithm is to find each root’s edge, or the transition between the root’s body and the background or root hairs using relative pixel brightnesses.

The algorithm is run on each candidate pixel. Starting at the candidate, the algorithm sends out 8 “probes” in all directions. Each probe steps in their initial directions as well as their relative right and left. They cannot move right or left twice in a row. This forces them to move in their initial direction while providing some leeway for different paths and root curvature.

These probes are used in future stages as well.

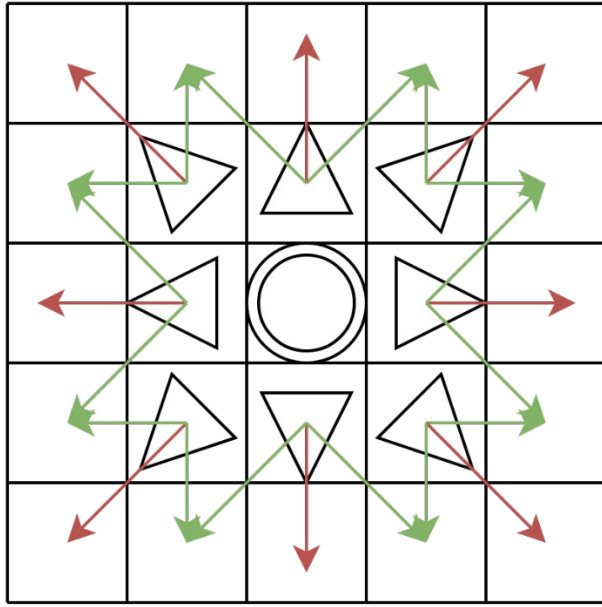


Figure 10: Shows which directions probes can move. Probes and their directions are represented as triangles. The red arrows show their main direction, and the green shows how they can turn.

At the beginning of each step, the algorithm moves copies of each probe forward, as well as left and right if possible. The brightness of the pixel at each probe's position is then checked against a threshold. If the brightness is less than the threshold, the pixel is considered to be an edge, and is removed from the list of candidates. The probe is also removed from future steps. This continues until no probes are left or the search limit is reached. It is implemented in C++ and runs in parallel for speed.

The threshold starts as a multiple of the candidate's brightness and is updated each step to be a multiple of the darkest pixel currently encountered. The darkest pixel is the best representation of a "true" edge and makes false edges significantly less likely. It also makes the algorithm more likely to miss valid edges, but there are multiple chances to find each edge since the function is run multiple times. There is also another threshold that is also a multiple of the initial brightness. It represents the lower limit for edge brightness and allows the algorithm to

still remove some candidates even if there are very dark pixels nearby. The user can adjust these 3 thresholds as well as the search limit.

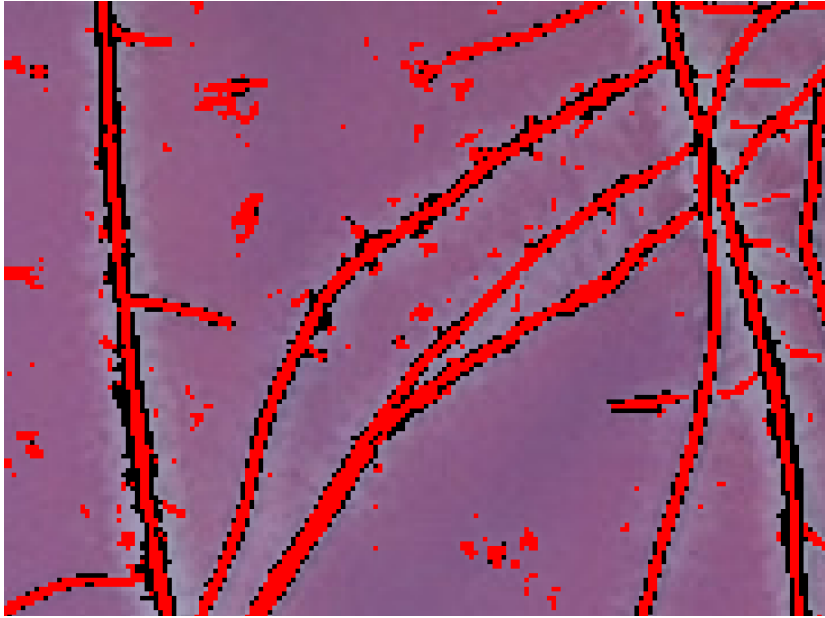


Figure 11: The results of the edge finder algorithm. Roots are in red, removed labels are in black.

4.1.4 Texture Matcher

The fourth stage is meant to remove obvious background using simple color information. It first creates 11x11 pixel texture images centered on each candidate pixel. It then generates features based on the textures and uses a support vector machine (SVM) to determine if the pixel is background or foreground. The SVM is trained on example foreground and background textures that are provided by the user. If a pixel is found to be background, it is removed from the set of candidates.

The features are made from a 3D color histogram, which is a representation of the texture's color frequency. To compute the histogram, the image is first converted into the LAB color space. L represents perceptual brightness, and A and B represent color. Unlike the RGB color space, the LAB space is designed such that if two positions are near each other, they are

also perceptually similar. This makes the histogram more effective since the values that are binned together will have similar appearances. The histogram is 3D, meaning that each combination of L, A, and B have their own bin. L is given 8 bins and A and B are given 16 bins each, between a range of [58, 203) since the A and B space is not fully used. This results in a histogram with a total of 2048 bins, and the normalized bins are given to the SVM as features.

The user can provide any number of foreground and background examples by right and left clicking respectively. Each time the image is clicked, the program will save an 11x11 texture centered around the click. It will then be shown on the control panel, where it can be removed by double clicking. The SVM is retrained and rerun whenever the user presses Enter. The matching process is parallelized to minimize execution time.

The matcher generally requires at least a handful of samples, which can take the user several minutes. However, the examples are saved and will be loaded for future images from the same dataset. Therefore, the first image of a dataset will generally take a bit longer than future images, but the cost to provide samples becomes small when averaged over the entire dataset.

This stage is primarily useful for removing background pixels that have significantly different colors than any root. It is not intended to work in complex or nuanced situations. If the image is grayscale or the matcher is otherwise ineffective, it should simply not be used.



Figure 12: Illustrates the effects of the texture matching stage. Black pixels have been removed.

4.1.5 Fill Small Holes

The purpose of the fifth stage is error correction. The previous stages can leave roots with gaps in their labels. This would be inaccurate in itself but would also negatively affect the results of future stages. Gaps should not mindlessly be filled however, since it is possible for overlapping roots to form holes legitimately. Therefore, this stage finds all small gaps, and then compares their average brightness to the average brightness of their border. If the hole has a similar brightness, then it is filled.

The program first fills all small holes using scikit-image's `remove_small_holes` function. Then it gets the holes by themselves by performing the logical XOR of the original and filled labels. The program obtains the border of each gap by using scikit-image's `binary_dilation`

function and logically XORing it with the original. The brightness threshold is calculated from a multiple of the mean of the border. If the mean of the hole is greater than the threshold, the hole is filled.

The user can adjust the maximum hole size and the brightness threshold multiplier. The maximum hole size determines the largest gap that the algorithm will attempt to fill. Both values should be experimented with for optimal results on each dataset.

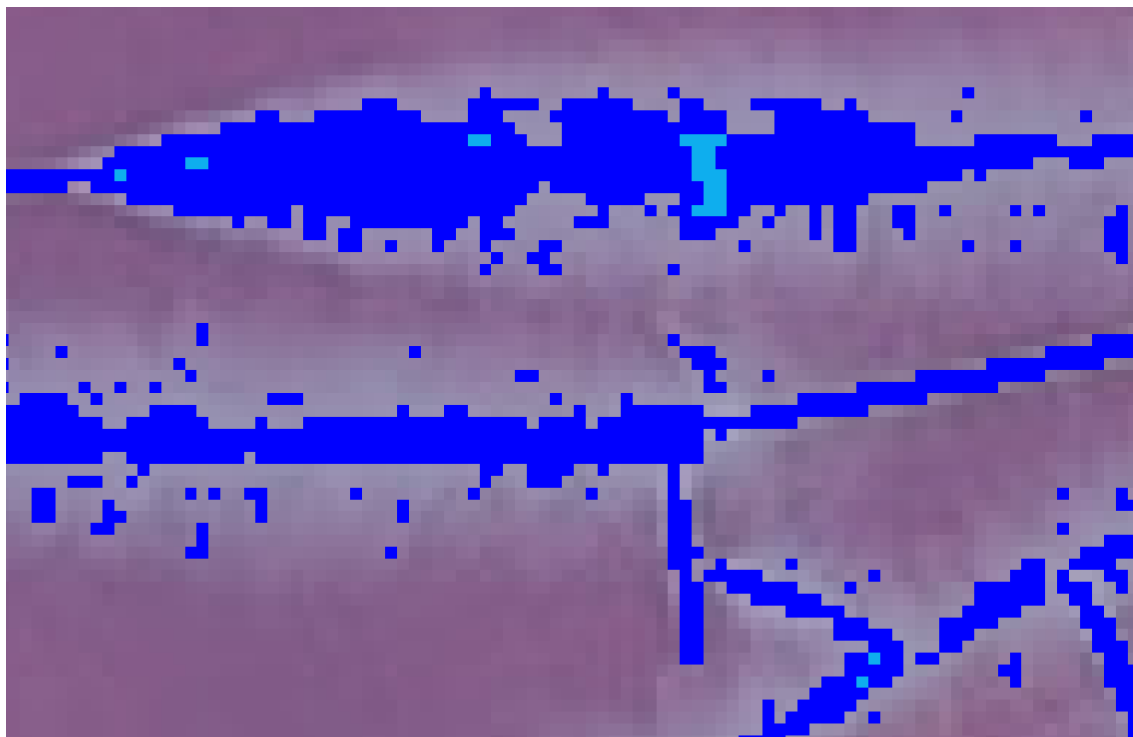


Figure 13: Shows the effects of the fill small holes stage. Roots are labeled in blue and filled holes in light blue.

4.1.6 Connect Separated Roots

The sixth stage also exists to correct errors. One of the main issues with the gaussian filter and edge detection algorithms is their tendency to not label pixels when a bright root intersects a darker one. The gaussian filter compares pixels to the local brightness, so a dark root can be overwhelmed by a bright one in close proximity. Likewise, the edge detection algorithm

may correctly find the edges of the bright root, but these edges coincide with the darker root. Both issues have the effect of disconnecting root labels, which would have a very negative impact on the next stage. This stage attempts to repair the labels using the roots' structural information.

Roots tend to travel in straight lines, and only make gradual curves. To help identify straight segments, the program first skeletonizes the labels. This removes unnecessary detail and makes it easier to determine a root's general direction. Then the program finds root ends by convolving the labels with a 3x3 kernel of 1s with a 10 in the middle. A skeleton endpoint is defined as a label with exactly one neighbor. Therefore, if a convolved pixel is exactly 11, then that point is an endpoint. For each endpoint, the algorithm traverses through all possible paths. The skeleton normally has only one path, but there are multiple options at each branch. The program does not know which of these represent a contiguous root, so it tries them all. Each segment is then tested to find the first subsegment that is straight enough. A subsegment is considered straight if it only travels in one main direction. They are allowed to deviate in only one adjacent direction.

The program now tries to reconnect each potential broken end. It looks for a path from the endpoint to a separate label group. A label group is considered separate if it cannot be reached by a flood fill of a range slightly greater than the maximum search limit. This prevents the algorithm from making a connection inside the same root, while still working on connected regions that are far apart. Very small label regions cannot be connected to, as they are not substantial enough to represent a valid target. The path needs to be in a similar direction to the straight segment. Each step on the path must also have a similar brightness to the segment, to prevent connecting legitimate root ends.

The program searches for a path using the probes from the edge detection stage. It moves copies of them forwards at every step until a path is found or the search limit is reached. Probes are removed at each step if they deviate from the segment's angle by more than 22.5 degrees. They are also removed if they have a lower brightness than the minimum threshold, which is a multiple of the median of the segment's brightnesses. If a connection is found, the algorithm adds it to a list of path candidates and continues for a couple more steps to ensure it finds the best path.

The angle of each candidate path is compared to the angle of the parent segment, and the path with the closest angle is chosen. The program connects the chosen path by making a footprint of the labels 8-adjacent to the segment's end point. Then it sets the labels in the footprint for each point on the path. This method is a good approximation of the reconnected root's actual width.

The entire algorithm is repeated until it stops finding connections. This allows sections with multiple broken sections to build off each other. The straight segment detection is implemented in Python, but the more intensive path search algorithm is implemented in C++. This makes the algorithm fast enough to be run multiple times.

The user can adjust the search distance, brightness threshold multiple, and the minimum length a straight segment needs to be. The search distance needs to be long enough to make proper connections, but being too long can cause more errors. The algorithm is also exponential in search length, so it should be kept to a reasonable size. The brightness threshold should be experimented with for each dataset for optimal results. The minimum straight segment length determines how much structural information is required to start looking for a connection. Higher

values will make fewer mistakes, but will also miss valid connections, while the opposite is true for lower values.

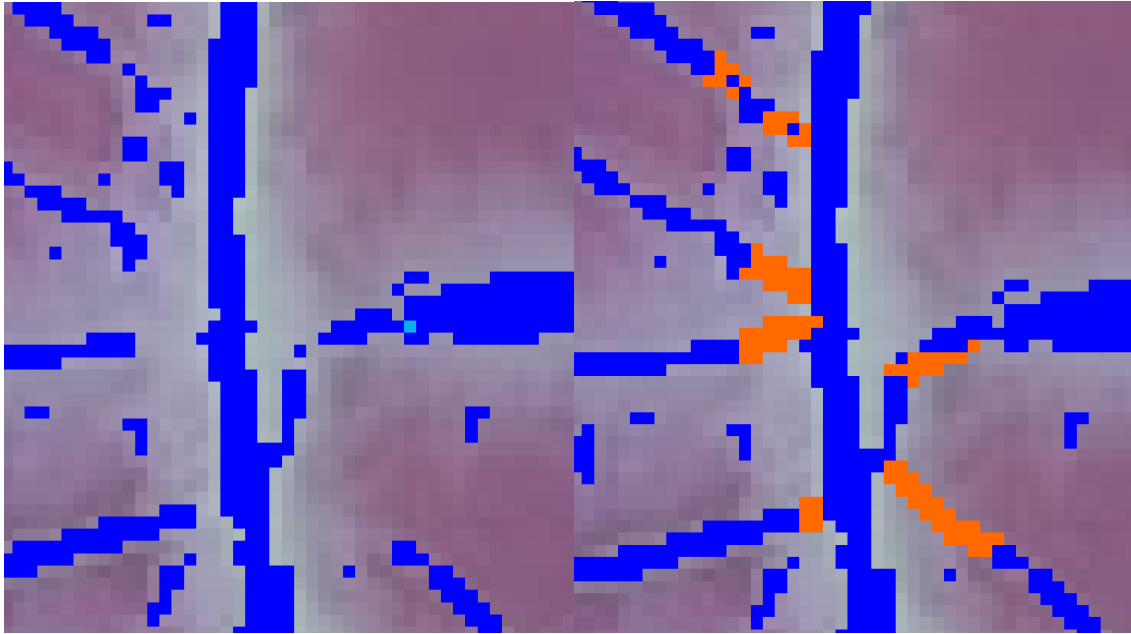


Figure 14: On the left, the disconnected roots from the gaussian filter and edge detection stages. On the right, the orange labels represent the reconnected segments.

4.1.7 Remove Small Objects

The seventh stage is used to remove any small label groups. If the root system is largely connected, this can remove almost all false positives by itself, and it is still useful even in disconnected root systems. In any root image, there will be small, disconnected labels far away from any actual root. These are caused by noise or other effects. It is very difficult for a human to notice and remove them all, since they are so small and numerous. This stage is a simple way to completely prevent these issues with minimal drawbacks. The user can adjust the minimum region size.



Figure 15: The results from the remove small object stage. Removed labels are shown in black.

4.1.8 Manual Correction

The eighth and final stage is meant for manual correction. This stage is the main focus of the program, since it is the only one that can achieve human level accuracy. The previous stages were only meant to quickly provide the user with a good starting point. This stage provides the user with 3 main tools to make corrections.

4.1.8.1 Delete in Radius

The first tool is the ability to delete labels in a radius by middle clicking the mouse. This is useful for removing incorrect labels in areas with minimal actual roots. It is also useful for “smoothing” a root’s edges by repeatedly deleting along the edge. The default radius is small, but it can be made bigger by holding the Shift or Control modifiers.

4.1.8.2 Delete Segment

The second tool is designed to remove root segments. The user can right click a label, and the program will identify the segment that pixel belongs to and remove it. A segment is a portion of the skeleton of the labels that does not contain any branches. It also includes all labels that are closer to the segment than any other skeleton.

When the user clicks, the program skeletonizes all labels. It then uses a flood search to find the closest skeleton to the start position. However, since the neighbors are 8-connected, the search cannot simply return the first skeleton encountered. Instead, it must record the minimum distances to reach each visited skeleton and keep searching until the search depth exceeds the closest skeleton. Once the program has the nearest skeleton pixel, it traverses the adjacent pixels until it reaches a branch or an endpoint. This results in the selected segment's skeleton. The program then needs to find all labels that are closest to the segment. It repeats the nearest skeleton algorithm for all labels and removes any that are closest to the segment. This tool is largely implemented in C++.

This algorithm is useful for removing long sections of labels. It can easily remove erroneous "bridges" between two valid roots. It is also effective when there are valid labels nearby that are too close to delete by radius. It works well with the root connection stage since most mistakes that stage makes can be removed without taking any extra time.

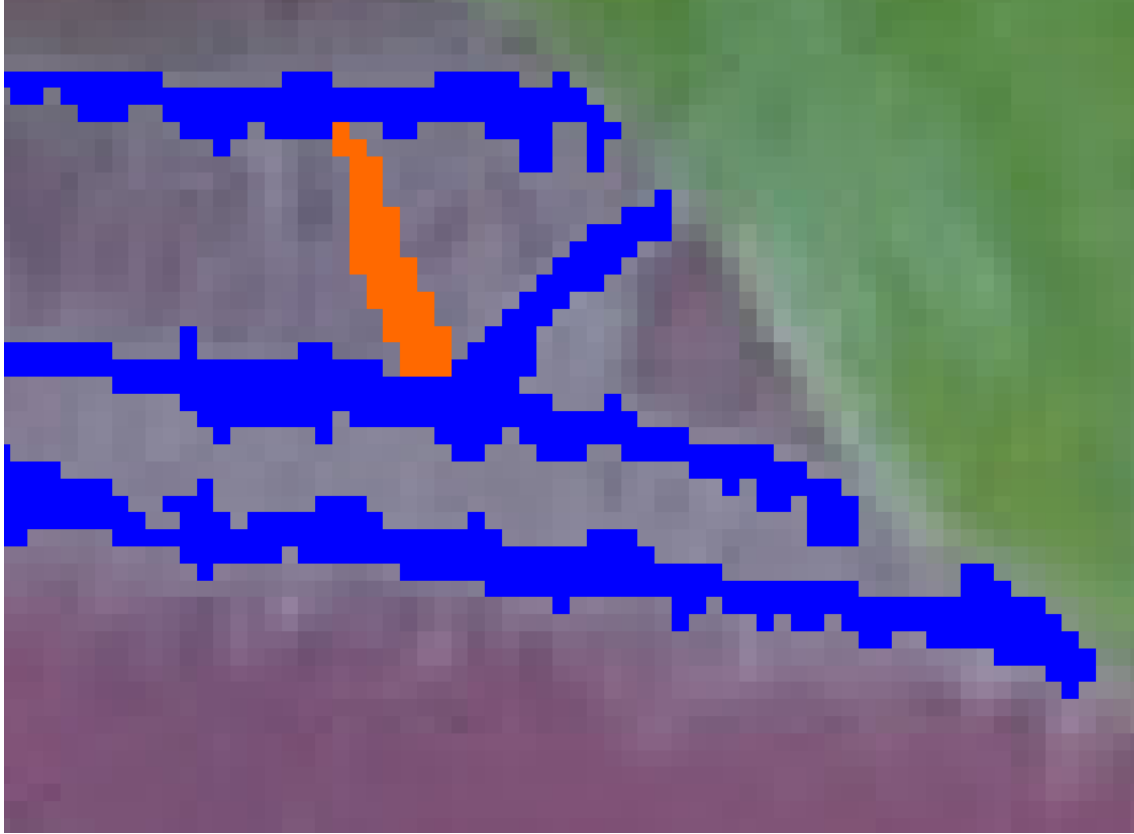


Figure 16: Result of deleting an erroneous connection by right clicking it anywhere in the region. Removed labels are in orange.

4.1.8.3 Add Segment

The final tool is designed to add labels. It allows the user to label roots by just clicking their start and end positions. The user marks the start position by left clicking, and the end position with another left click. The algorithm finds the best path from the start to the end and can track a root's curve and its thickness.

The algorithm uses probes to find the best path. The probes used previously were intended to be able to curve slightly but were largely fixed in direction. The probes used here can change their direction based on their recent moves. If a probe repeatedly makes turns, its direction gets updated to be the turned direction, allowing it to turn more in that direction. This

approach forces the probes to travel in the same general direction while still being able to track a curving root.

The algorithm uses “fast” and “slow” probes at the same time. Fast probes can turn in the same direction multiple times in a row, and slow probes cannot. This gives the fast probes a smaller turning radius. The algorithm returns the first probe that finds the endpoint. Sometimes a brighter root will intersect the root that the user is trying to label. The fast probes are more likely to deviate, while the slow ones will stay the course. On the other hand, sometimes a root curves too quickly for the slow probes to keep up, while the fast probes can. It is very rare for a probe to deviate and still find the endpoint before a correct probe, so this approach expands the number of situations the algorithm can handle.

The algorithm works by creating probes in all initial directions at the start position. It then makes copies of them move in all possible directions. After every step, the fast and slow probes are each pruned to keep the top 500 with the brightest path. This gives the probes the freedom to explore a bit, while preserving linear complexity.

This approach works well because the probes are restricted in how they can move. However, at the beginning of the search, there are probes in all directions with very few restrictions. If a probe leaves the root and finds a brighter one close by, or the root is brighter in the opposite direction of the endpoint, the correct probes will quickly be pruned. To combat this, if the algorithm fails to find the endpoint within the search limit, it records the probes’ initial positions and blocks them. The search is run again, but if any probes enter the blocked positions they are immediately removed. If the search fails to find anything after 10 tries, it returns with an empty path. This allows the search to organically try multiple search directions without

artificial limits which may incorrectly prevent the actual solution. The search is completely implemented in C++ to compensate for the amount of computation required.

Once a path is found, it is smoothed with a cubic spline. The start and endpoints of the path are given a large weight to prevent the spline from deviating. The spline is allowed a squared error of 1.5 times the length of the path. This value was empirically found to allow the spline to smooth small deviations without leaving the root altogether.

The thickness of the root is determined using a small flood fill on each pixel on the path. If a pixel is greater than 99% of the brightness of the path's pixel, it is labeled as a root. The flood is restricted to a depth of 2 because it is unlikely for a root to be thicker than 5 pixels.

The algorithm overall works well in many situations. However, sometimes it fails to find a path, and sometimes it finds an incorrect path. In these situations, the user can press Shift and the program will simply draw a line between the two points. The points on the line will still have the thickness algorithm applied.

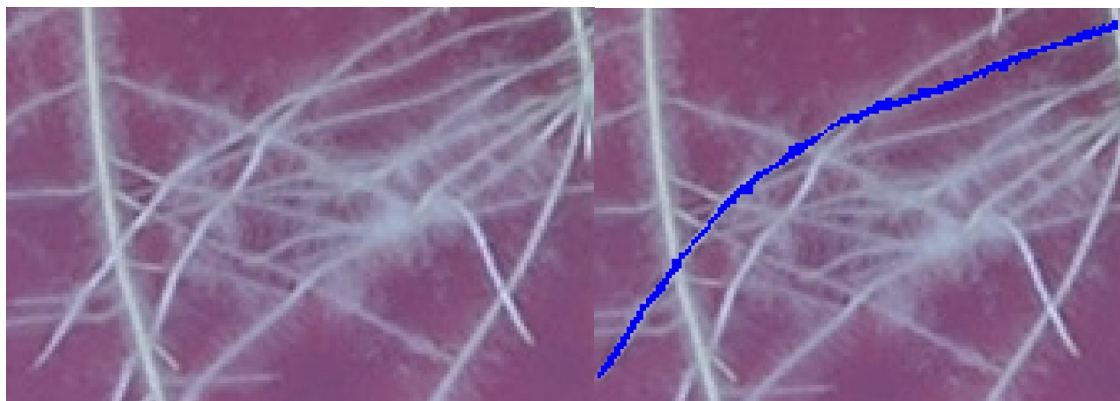


Figure 17: On the left, an unlabeled overlapping root system. On the right, a root's path traced using the add segment tool.

4.1.9 Neural Network

Neural networks are currently the state of the art for most image processing tasks. However, they cannot be used without training on high accuracy labeled data. They are also not currently capable of matching human level accuracy. To mitigate these issues, this program uses a technique called active training.

Active training is a method to reduce the amount of effort required to make a neural network useful. Normally, a network must be trained on a very large amount of labeled data and requires a lot of hyperparameter optimization. This is because the networks are intended to be used by themselves, so they must be as close to perfect as possible. Active learning lowers the bar by always intending for human oversight. This means the network becomes useful as soon as it becomes more accurate than the original program. This allows the network to start working with a very small amount of training. The network can also be retrained as more data becomes available. As it repeatedly improves, it will save more time, and more labels will become available faster.

4.1.9.1 Network Structure

The network used in this project is a convolutional U-Net with 5 encoding and decoding stages. This means any input images should have heights and widths divisible by 32 to maintain their dimensions. Each stage of the encoder is comprised of a dense convolutional block of 3 layers, followed by a max pooling layer and a batch normalization layer. A dense block contains skip connections from each layer's output to each subsequent layer's input and are concatenated together for the block's final output. This allows the network to make multi layered

computations without losing the intermediate steps, and allows the gradient to flow back more effectively, which aids learning.

Each convolutional layer in this network's dense blocks is followed by a batch normalization layer. Batch normalization is an anti-overfitting technique. It works by normalizing a layer's output by subtracting the batch's mean and dividing it by the batch's standard deviation. This compresses the output to be within a much smaller and more consistent possible range, which reduces the effect that very large or small outputs can have on future layers. The mean and standard deviation are calculated during training, then saved and loaded when performing inference. This vastly improves the network's ability to generalize, at the expense of computation time.

The dense convolutional layers have a 25% chance of their outputs being zeroed out completely each training step. Dropout reduces the effective size of the network during training, which makes it more resistant to overfitting. Additionally, it forces the network to learn more redundant and general rules, since it cannot rely on any particular connection, as any layer can be removed at random. Dropout is not applied to the first layer, since that would potentially remove all input for future layers.

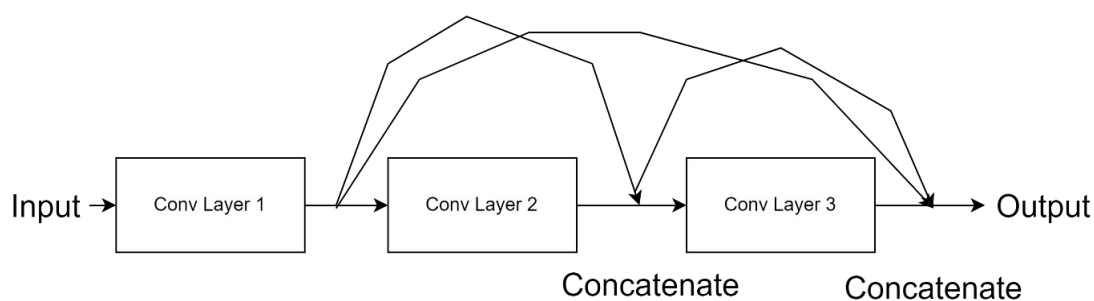


Figure 18: The dense convolutional block used in the network. Each convolutional layer is also followed by a batch normalization and dropout layer before concatenation.

In between the encoder and decoder there are two convolutional layers, each followed by a batch normalization layer. This section of the network is the most spatially compressed, so a convolution here “sees” more of the image than anywhere else. These layers take advantage of this by using a kernel size of 7x7 instead of the standard, which allows the network to take a massive amount of global information into account.

Each stage of the decoder is comprised of a 2D convolutional transpose layer followed by a dense convolution block. A convolutional transpose layer is the inverse of a max pooling layer and allows the network to perform upsampling with learned weights. The layer uses a 4x4 kernel, which is important to be a multiple of its 2x2 stride for numerical reasons. The decoder produces outputs with the same size as the original image. The output layer uses a 1x1 kernel to produce a binary output with a sigmoid activation function, where each value represents the predicted probability of that pixel being a root.

The network uses a Dice loss function. The Dice coefficient is a measure of accuracy. It is calculated by dividing double the number of true positives by sum of the number of false positives and false negatives. This gives a number between 0 and 1. The Dice loss is the Dice coefficient subtracted from 1. This is also a number between 0 and 1, but now a lower score is better. It essentially gives the proportion of correctly labeled root pixels compared to the number of missed roots and incorrect root predictions. It spreads the loss evenly when the network makes many mistakes, and heavily penalizes errors when the network is mostly correct. Using a loss function like Dice is important, since most of the image is of the background, so the network could easily get 95% simple accuracy just by never predicting any roots.

4.1.9.2 Network Integration

If the network is not trained yet, the program acts in stages like previously described. If it has been trained, it will be used to make predictions when the image is loaded. The predictions are also post processed with the root connection and remove small objects algorithms. The root connection algorithm generally makes more improvements than mistakes, and many mistakes take the same amount of time to fix because of the segment deletion tool. Remove small objects is run with a max size of 10. Running it like this will have very few negative effects but will remove small specks that the user may miss otherwise. If the user likes the predictions, they can press Enter to be taken directly to the manual correction stage. If the predictions are subpar, the user can press N to use the program as normal. The user must handle training on their own, as it requires far more computational resources than inference. A mid-range CPU can make a prediction in a few seconds, but doing a decent amount of training would take days at least.

CHAPTER 5: EVALUATION

To evaluate performance, the program was compared against RootNav and another semi-automated tool called saRIA (Semi-Automated Root Image Analysis) [10]. These programs were chosen because they are the best semi-automated tools currently available. Additionally, this program was tested without its neural network, with the network trained on 6 samples (4 train, 2 validation), and trained on 12 samples (8 train, 4 validation). Each networks' outputs were also tested without any user correction. The programs were evaluated on 3 datasets.

5.1 Datasets

The first and most difficult dataset arranges its images in sets of 7. The set is a time series tracking the growth of 8 plants, with pictures taken every 2 days. The images have a resolution of 1664x1600 pixels. The plants are close together, resulting in a lot of overlapping. The older plants also have leaves that obscure a significant portion of the base of the roots. The leaves cast shadows that make some parts of the image locally darker than others. The red background has a slight white tint and some white specks that can be difficult for algorithms to differentiate from the roots. The center ring and sides of the case are also relatively bright, causing the same issue. Since the images come in sets, the neural network was only trained on 6 sets of images (28 images train, 14 images validation). However, the programs will only be evaluated on the final images of a set, as these plants are the largest and most difficult to label.



Figure 19: A sample image from the dataset. The image was taken on the last day of the experiment, making the plants 12 days old. Note the leaves obscuring the base of each plant, as well as the highly overlapping secondary roots.

The other two datasets are open source from GigaDB (Andrew, F. et al.). The first consists of rapeseed seedlings grown on agar plates. The images in this dataset have a resolution of 2500x1916 pixels, but since most of each image does not contain roots, their height was cropped to the range of [70, 2182), and their width to [380, 1500), for a final size of 2112x1120. Each image is of a single plant on a blue background. The main difficulty is the smudging and fuzz around the top of the plant, which can confuse many algorithms.

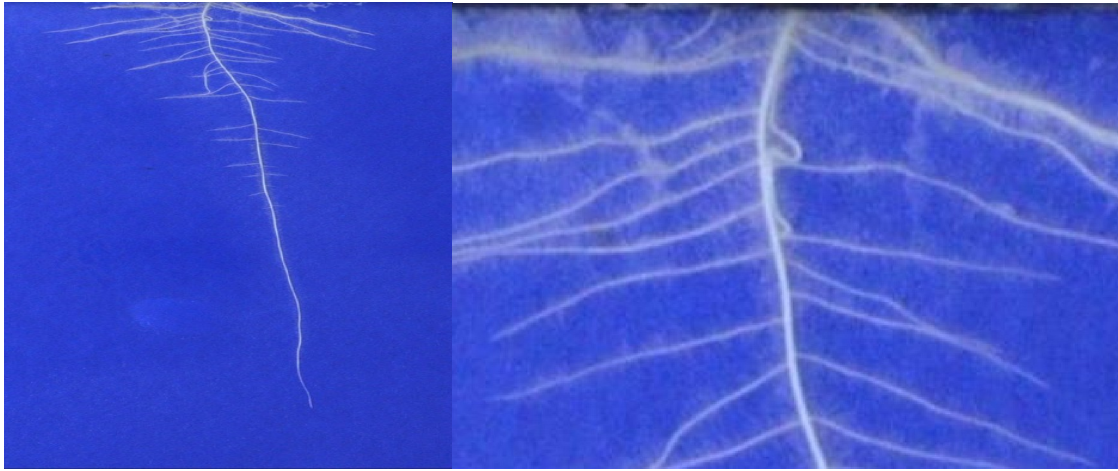


Figure 20: On the left, a sample from the cropped rapeseed dataset. On the right, a closeup of the top of the root where smudging can be seen.
Source: Adapted from [11]

The other dataset consists of *Arabidopsis* grown on agar plates. The images have a resolution of 1024x1024 pixels. They are of a group of 5 plants, spaced far apart. The dataset is in black and white, which mainly makes it tricky to tell the leaves apart from the roots. Some of the roots are also very dark, making it ambiguous whether they should be labeled. Both datasets have existing ground truths, but they had to be discarded due to inaccuracies.

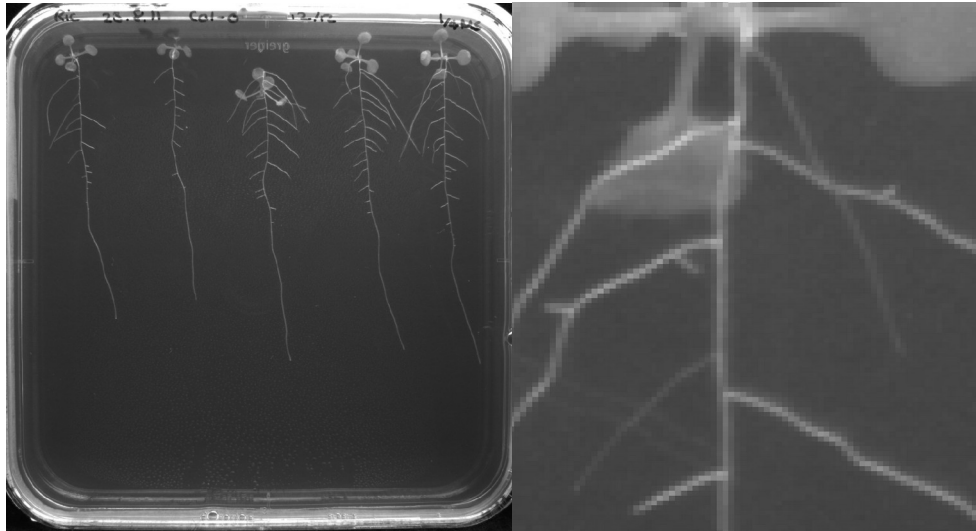


Figure 21: On the left, a sample from the Arabidopsis dataset. On the right, an example of a couple of very dark roots.

Source: Adapted from [11]



Figure 22: An example of the existing labels from the rapeseed dataset. Note the unlabeled offshoots and how the labels lose track of the roots when they curve.

Source: Adapted from [11]

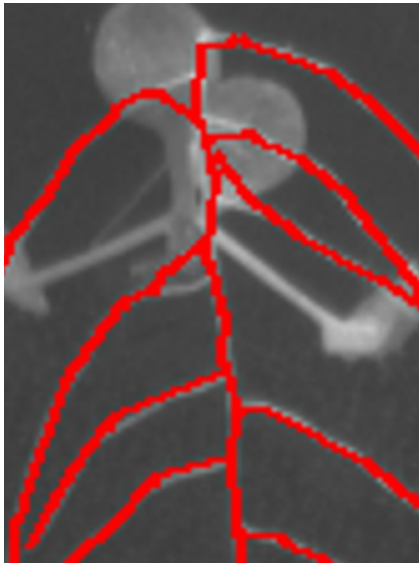


Figure 23: An example of the existing labels from the Arabidopsis dataset. They are more accurate than the rapeseed dataset, but they still miss roots and lose track when the roots curve.
Source: Adapted from [11]

5.2 Metrics

The most important metric the programs were graded on was accuracy. If the accuracy is subpar, nothing else matters. This was quantified using the dice coefficient. The next metric was speed. It is already possible for humans to perfectly label plants if they spend enough time, so accuracy alone is not a sufficient metric. Small accuracy losses are acceptable if enough time is saved. Finally, the programs are judged on adaptability. Each program should perform similarly on each dataset. If the software is not consistent, taking the time to use it on a new dataset is inherently risky.

5.3 Results

Table 1: The average Dice coefficient of each combination of software version and dataset.

	RootNav	saRIA	Project	Neural Net 6	Neural Net 12	Neural Net 6 Auto	Neural Net 12 Auto
Main Dataset	N/A	.6584	.9313	N/A	.9769	N/A	.9702
Arabidopsis	.6619	.7350	.9420	.9721	.9893	.9548	.9829
Rapeseed	.6279	.4539	.9272	.9529	.9826	.9407	.9780

Table 2: The average time in seconds it took to label each combination of software version and dataset.

	RootNav	saRIA	Project	Neural Net 6	Neural Net 12
Main Dataset	N/A	199	865	N/A	403
Arabidopsis	379	164	245	148	108
Rapeseed	487	106	510	197	145

Overall, this project had significantly higher performances than the others. Its accuracies were much higher in all cases, and its speed was only lower than saRIA in the main and rapeseed datasets. In those cases, saRIA's outperformance was because it lacked the tools needed to make the labels more accurate, so it ended sooner since there was nothing to do. This project was also extremely consistent, having very similar accuracies across datasets. However, there are a few potential caveats that may have biased the score in its favor.

5.4 Potential Issues

Firstly, since there were no quality ground truths for any of the datasets, new ground truths had to be produced. This was done by using this program with the most trained version of its neural network, and spending much more time than normal manually ensuring everything was completely correct. This does not guarantee 100% accuracy, and it is likely the neural networks obtained a higher score than they should have because a significant portion of the network's predictions were kept in the ground truths. However, since the final labels were approved manually, they should still be largely accurate, meaning this is not enough to explain the massive accuracy differences. The project's non-networks scores were also likely hurt, which makes the results more balanced.

The second caveat is that RootNav has lower scores than it probably should. This is because its labels are just slightly off, meaning it has the “spirit” of the roots, but not the exact values. However, it did still make some significant errors, like missing some offshoots completely. RootNav also did not receive a score for the main dataset, as it was so overwhelmed by its complexity that it would have likely taken at least an hour to label a single image and would still have poor accuracy.

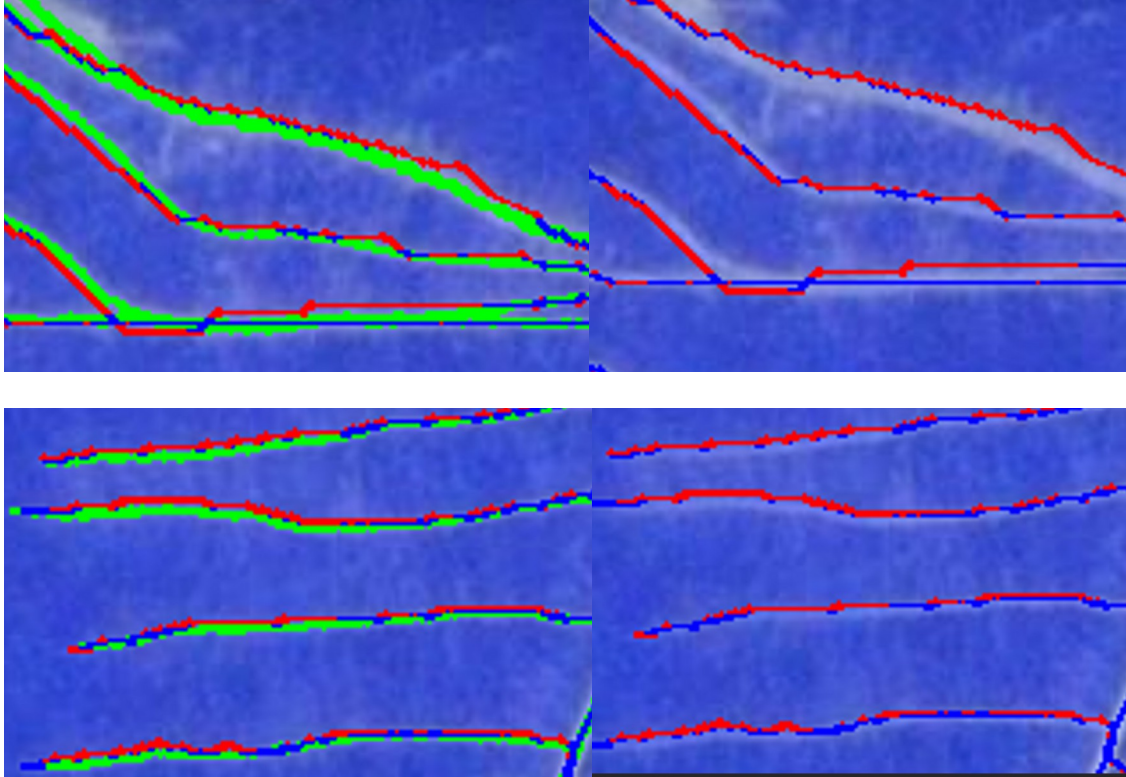


Figure 24: Illustrates the type of errors made by RootNav. The blue labels represent RootNav and the ground truth in agreement. Red labels represent where RootNav predicted a root, but the ground truth did not. Green labels represent where the ground truth had a root, but RootNav did not. The left side has the same image without the green labels, to show the underlying root. The Dice coefficient is purely determined by the number of blue labels, despite RootNav's errors being so close to accurate.

Source: Adapted from [11]

5.5 Program Strengths

There are a handful of reasons why this program had superior performance compared to the others. Firstly, the other programs only had a few algorithms and ways for the user to influence those algorithms. There are a huge number of variables between root datasets, including plant species, image resolution, background color and brightness, number of plants and their proximity, occluding objects, and more. These variations cannot be accounted for by a few controls. The user needs to be able to provide much more information to achieve high accuracy, and this program facilitates that, while also allowing them to ignore ineffective stages to preserve speed.

A second reason is that the other programs do not have very good user experience. Neither RootNav nor saRIA have any form of undo or redo, so if the user makes a mistake, they must either live with it or start from scratch. They also make it difficult to see the program's state and the underlying root image at the same time. RootNav can toggle the image and its predictions, but it is hidden away in a menu. saRIA cannot remove its labels at all. It also does not allow the user to zoom in, and color images are always shown in grayscale. These issues and more drag the user down and causes them to make and not fix mistakes. They also wear the user out, which would cause them to get tired and make even more mistakes. The scores in this experiment are likely higher than what would be achieved in practice, like how the existing labels had to be thrown out due to significant errors.

5.6 Network Training

This project's neural networks were trained on a V100 GPU for 2 hours with a batch size of 2. The networks were very resistant to overfitting, even with such small amounts of data. The Arabidopsis set overfit the most, likely because there was no color information. The 6 samples version had to be terminated early due to lack of progress. All networks achieved higher validation scores and resisted overfitting with more samples. The main data was run for a total of 12 hours, and improved consistently the entire time, and always had lower validation loss than training loss. The network's accuracy also correlated with faster and more accurate manual corrections. These results suggest that training a useful network requires little initial investment and will produce even better results as more training data is provided. Additionally, while the networks are more accurate after human correction, it is not an extreme difference. This suggests

that the networks could already be used as fully-automated software after training on only a handful of labels.

CHAPTER 6: FUTURE WORK

There are a few areas where this project could be improved. The most important aspect would be a better root “smoothing” tool or stage. Currently, fuzzy roots often have a lot of strange offshoots that need to be removed. Removing them involves repeatedly using the radius delete tool along the length of the root at the proper spacing. If this could be done faster or automatically, it would probably make labeling take half as long. Another area is root analysis. Currently, the program just produces a segment map of the roots. It does not actually provide any direct analysis. This is relatively simple compared to segmentation, but it is a sorely lacking feature. Finally, the root connection algorithm could be improved. It works decently well, but there are always many sections that it misses, and others that should not be connected. This would not be as impactful as root smoothing but would still be very helpful.

CHAPTER 7: CONCLUSION

This project set out with the intention of creating a highly accurate root labeling tool that required minimal human time. By many metrics, it appears to have been a success. The program works well initially, and even better once the neural network has been trained. Training the network only requires a handful of samples to be useful. As technology improves, neural networks will continue to become more prominent and effective, but they will still need high accuracy labels to train on. This program can provide those labels, while also lowering the bar

for accuracy by allowing manual correction. There is no other semi-automated tool that currently exists with the same focus on user experience, control, and accuracy.

REFERENCES

- “The Development of Agriculture.” *Education*, National Geographic. Available: education.nationalgeographic.org/resource/development-agriculture/.
- Norouzi, Omid, et al. “In Vitro Plant Tissue Culture as the Fifth Generation of Bioenergy.” *Nature News*, Nature Publishing Group, 23 Mar. 2022. Available: www.nature.com/articles/s41598-022-09066-3.
- Lobet, Guillaume, et al. “A Novel Image-Analysis Toolbox Enabling Quantitative Analysis of Root System Architecture.” *Plant Physiology*, vol. 157, no. 1, 19 July 2011, pp. 29–39. Available: <https://doi.org/10.1104/pp.111.179895>.
- Pound, Michael P., et al. “RootNav: Navigating Images of Complex Root Architectures.” *Plant Physiology*, vol. 162, no. 4, 13 June 2013, pp. 1802–1814. Available: <https://doi.org/10.1104/pp.113.221531>.
- Yasrab, Robail, et al. “RootNav 2.0: Deep Learning for Automatic Navigation of Complex Plant Root Architectures.” *GigaScience*, vol. 8, no. 11, 1 Nov. 2019. Available: <https://doi.org/10.1093/gigascience/giz123>.
- Mandal, Manav. “CNN for Deep Learning | Convolutional Neural Networks (CNN).” *Analytics Vidhya*, 1 May 2021. Available: www.analyticsvidhya.com/blog/2021/05/convolutional-neural-networks-cnn/.
- Rathi, Mukul. “Convolutional Neural Networks.” *Mukulrathi.com*, 4 Sept. 2018. Available: mukulrathi.com/demystifying-deep-learning/convolutional-neural-network-from-scratch/.
- Yasrab, Robail, et al. “RootNet: A Convolutional Neural Networks for Complex Plant Root Phenotyping from High-Definition Datasets.”, *bioRxiv*, 3 May 2020. Available: www.biorxiv.org/content/10.1101/2020.05.01.073270v2.full.

Zhang, Jeremy. "UNet Line by Line Explanation." *TowardsDataScience*, Medium, 18 Oct. 2019.

Available: towardsdatascience.com/unet-line-by-line-explanation-9b191c76baf5.

Narisetti, N. *et al.* Semi-automated Root Image Analysis (saRIA). *Sci Rep* **9**, 19674 (2019).

Available: <https://doi.org/10.1038/s41598-019-55876-3>

Andrew, F. P., Darren, W. M., Jonathan, A. A., Michael, P. P., Robail, Y., & Tony, P. P. (2019).

Supporting data for "RootNav 2.0: Deep Learning for Automatic Navigation of Complex Plant Root Architectures" [Data set]. GigaScience Database. Available:

<https://doi.org/10.5524/100651>